3_12_marzo

FTP - File Transfer Protocol
FTP lavora sulle porte 20 e 21, la 20 è usata per il passaggio dei dati e la 21 per i controlli/comandi

Bidirezionale: Lavora in entrambe le direzioni; permette sia di caricare (upload) che di scaricare (download) informazioni e file.

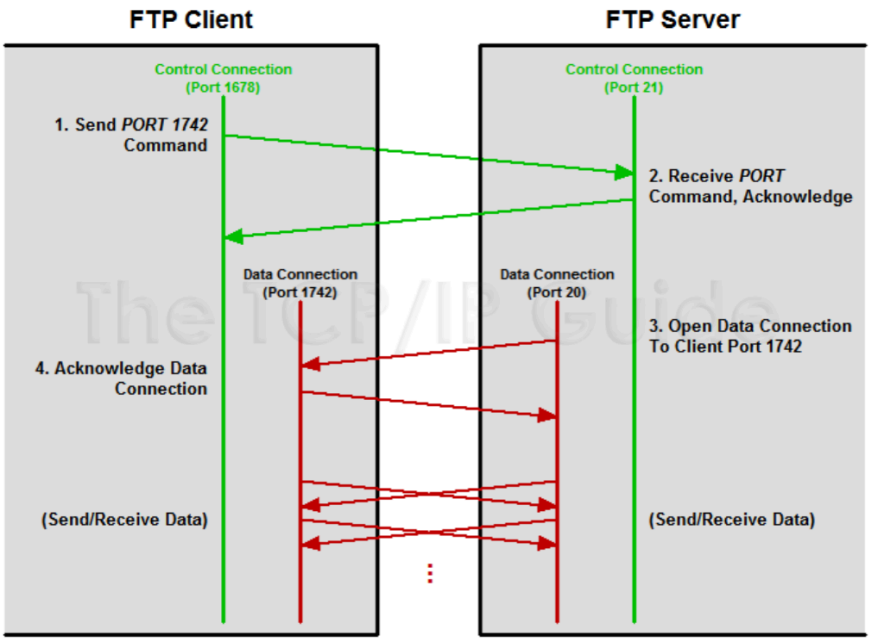
Sicurezza

- *Accesso*: Richiede quasi sempre un'autenticazione tramite username e password.
- *Sicurezza*: È un protocollo non criptato. Tutto il traffico, comprese le credenziali di accesso, viaggia "in chiaro" sulla rete

L'Architettura a "Doppio Canale"
A differenza dell'HTTP, l'FTP utilizza due connessioni separate per comunicare:

- *Canale di Controllo (Comandi):* Viene aperto sulla **Porta 21**. Serve esclusivamente per inviare i comandi (es. login, richiesta di un file, cambio cartella) e ricevere le risposte del server.
- *Canale Dati:* Viene aperto sulla **porta 20 solo quando serve** (quando si richiede il trasferimento effettivo di un file o la lista dei file in una cartella) e viene chiuso appena il trasferimento finisce.

Il vantaggio dei due canali: Poiché sono separati, il canale di controllo rimane sempre libero e reattivo. Se il trasferimento di un file molto grande si blocca sul canale dati, puoi usare il canale di controllo per dire al server "lascia perdere" (abortire l'operazione) e ricominciare, senza far crollare l'intera connessione



Il client apre una richiesta di connessione sulla sua porta (1742) il server apre la connessione sulla porta 21 e manda una ACK, dopo di ciò sulla la porta 20 apre la data connection per il client sulla porta 1742 (per il client) infine il client manda un ACK e da qui comincia lo scambio di file.

Differenza tra stateless e stateful

Stateful
Come funziona: Il sistema (di solito un server) "ricorda" chi è l'utente e cosa ha fatto nelle richieste precedenti. Crea una vera e propria **sessione** continua.
Ad esempio: Il protocollo **FTP** (che abbiamo appena visto: fai il login e la connessione rimane aperta in attesa dei tuoi comandi).

- Le vecchie telefonate tradizionali.
- I server dei videogiochi multiplayer online.

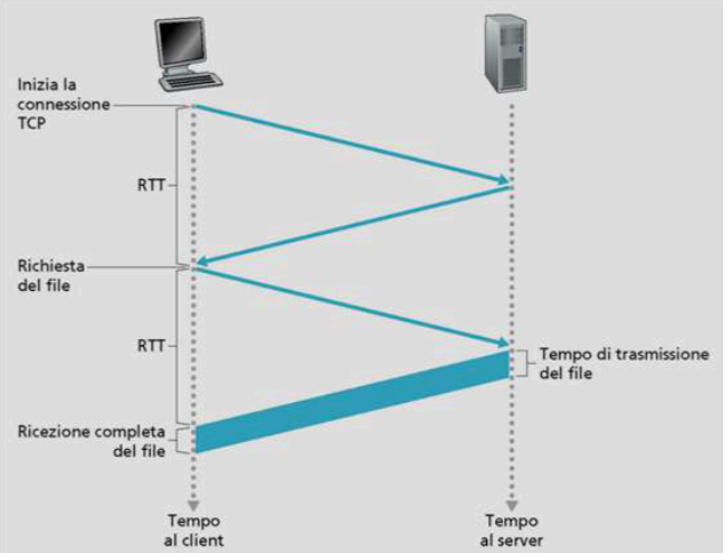
Stateless
Come funziona: Ogni singola richiesta (o messaggio) inviata dal client al server è completamente **indipendente** e isolata dalle altre. Il server elabora la richiesta, invia la risposta e poi se ne dimentica all'istante.
Ad esempio: Il protocollo **HTTP** (quello su cui si basa il web).

- Le API REST (usate dalle app sui nostri smartphone per comunicare con i server).

HTTP

HTTP nasce perché verso gli anni 90 si aveva bisogno di contattare un server in modo da recuperare documenti pubblici (si dovevano visualizzare le pagine dei siti web in pratica), quindi era inutile usare un protocollo come FTP (stateful), si può invece usare un protocollo *stateless* per migliorare e velocizzare i processi.

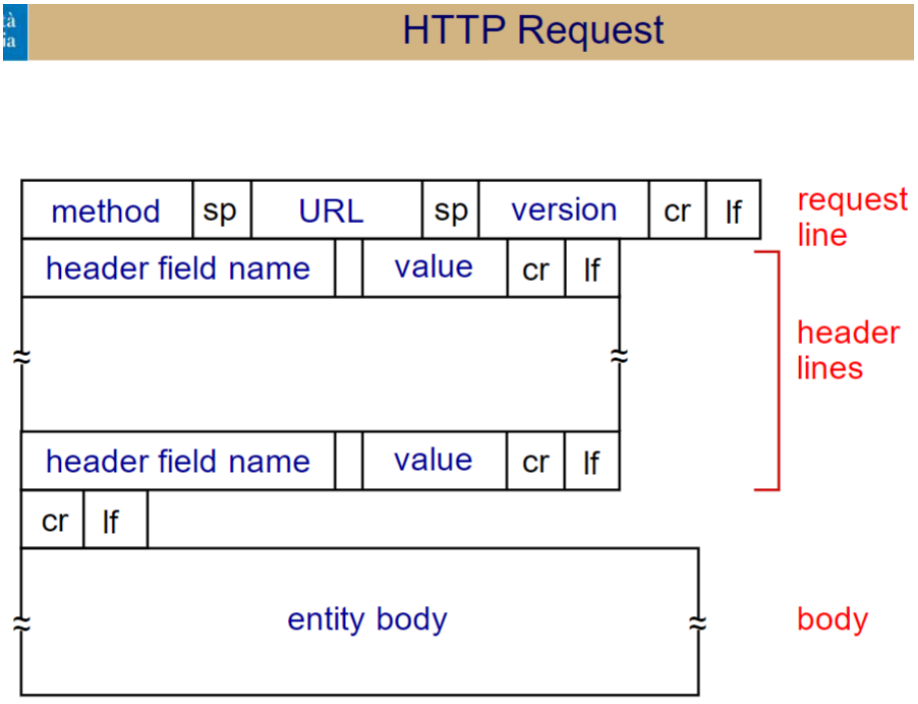
L'idea è quella di aprire una connessione TCP con il server e richiedere un documento che verrà così erogato del server.



Quindi il client manda una richiesta di connessione TCP e il server approva sulla porta 80 mandando un messaggio di ACK poi il client richiedere un determinato file e il server lo eroga

HTTP è quindi stateless, ovvero il server non mantiene informazioni sulle richieste passate

Come è formata una pagina web
Una pagina web è formata da un insieme di oggetti (che possono anche essere contenuti su diversi web servers), un oggetto può essere un file HTML, un immagine JPEG, un file audio ecc..., quindi una pagina web è composta da un file base HTML che include vari oggetti ognuno reperibile tramite un URL (Uniform Resource Locator)
Ad esempio: www.someschool.edu/someDept/pic.gif



Il messaggio si divide in tre blocchi principali:

Request Line (Riga di richiesta)
È la primissima riga del messaggio e contiene i "comandi" fondamentali. È composta da:

- **method (metodo):** Indica l'azione che il client vuole che il server esegua. I più famosi sono **GET** (usato per richiedere una risorsa, come una pagina web o un'immagine) e **POST** (usato per inviare dati al server, come quando compili un modulo di login).
- **sp (space):** Un semplice spazio vuoto che fa da separatore.
- **URL:** Il percorso specifico della risorsa richiesta sul server (ad esempio `/index.html` o `/immagini/logo.png`).
- **version:** La versione del protocollo HTTP utilizzata (di solito `HTTP/1.1` o `HTTP/2`).
- **cr lf (Carriage Return & Line Feed):** Sono i caratteri di "ritorno a capo". Indicano al server che la riga di richiesta è finita.

Header Lines (Intestazioni)

Dopo la riga di richiesta ci possono essere diverse righe di intestazione (gli *headers*). Servono a passare informazioni aggiuntive al server.

- Ogni riga è composta da una coppia: un **header field name** (il nome del campo, ad esempio `Host`, `User-Agent` o `Accept-Language`) e il suo **value** (il valore, ad esempio `www.google.it`, `Mozilla/5.0...` o `it-IT`).
- Ogni riga si chiude con il solito ritorno a capo (`cr lf`).

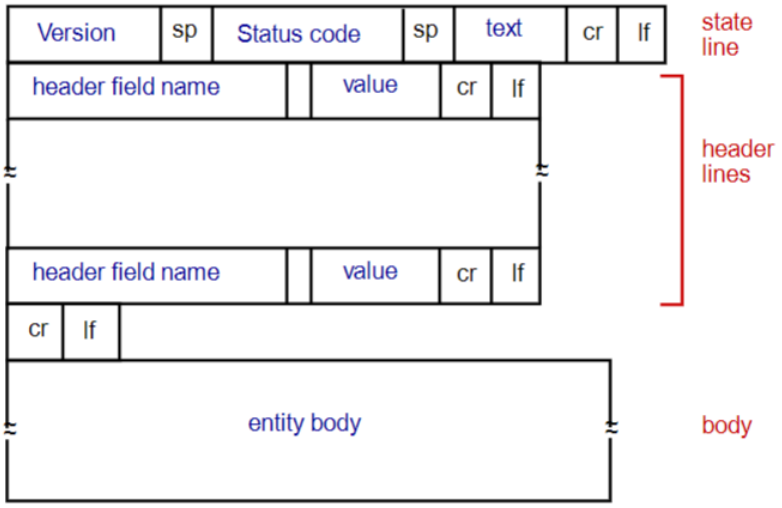
Il dettaglio cruciale: Guarda subito sotto l'ultima riga di header. C'è una riga formata **solo da** `cr lf` (un ritorno a capo a vuoto). Questa riga vuota è obbligatoria e serve come "separatore": dice al server che le intestazioni sono finite e che tutto ciò che viene dopo è il corpo del messaggio.

Entity Body (Corpo del messaggio)

È la parte finale che contiene il "carico utile" (payload) dei dati veri e propri che stai inviando al server.

- **Se usi il metodo GET:** Di solito l'entity body è completamente **vuoto**, perché stai solo chiedendo informazioni, non ne stai inviando (i parametri viaggiano nell'URL).
- **Se usi il metodo POST:** L'entity body conterrà i dati effettivi. Ad esempio, se stai caricando un file o hai scritto un commento su un forum, il testo del commento o i byte del file viaggeranno all'interno di questo blocco.

HTTP Response



L'HTTP response è identica alla request.

Method	Description
GET	Request to read a Web page
HEAD	Request to read a Web page's header
PUT	Request to store a Web page
POST	Append to a named resource (e.g., a Web page)
DELETE	Remove the Web page
TRACE	Echo the incoming request
CONNECT	Reserved for future use
OPTIONS	Query certain options

Cookie

Siti web e client browser usano i cookies per mantenere degli stati anche dopo la connessione (HTTP è stateless quindi non ha stati)
L'uso può essere ad esempio quello di identificare un utente sul sito dopo che accede per la prima volta, oppure per restare loggati in un sito anche dopo la chiusura, senza dover inserire ogni volta username e password.

I cookie sono sicuri perché non contengono codice eseguibile sul computer ma si limitano a raccogliere informazioni, sull'utente (ci possono essere dei rischi per quanto riguarda la privacy)

Versioni HTTP

Nel tempo si sono susseguiti vari aggiornamenti di questo protocollo, infatti un tempo si

apriva la connessione usando un solo canale in cui veniva mandato tutto, creando rallentamenti ed errori, poi si è aggiunta una pipeline che spedisce ogni oggetto usando un canale diverso, in questo modo per siti molto grandi se un canale non ha ancora finito di mandare il suo oggetto nel frattempo gli altri sono già arrivati e l'utente può comunque vedere qualcosa del sito.
In seguito sono stati aggiunti vari protocolli per la sicurezza

i protocolli sono retrocompatibili. Un protocollo si dice **retrocompatibile** (o *backward compatible*) quando una versione più recente o aggiornata è in grado di interagire correttamente con versioni precedenti dello stesso protocollo, o di gestire dati e dispositivi basati su standard più vecchi.